ENCRYPTED FILE-SHARING & VAULT SYSTEM

Project Report — Python / Flask / SQLite

Submitted by: _____

Register Number: _____

Department / College: _____

Academic Year: _____

Project Report: Encrypted File-Sharing & Vault System

Abstract

CipherVault is a Python web application for encrypted file storage and controlled sharing. Users register and log in, upload files that are encrypted before disk storage, download their own files, create expiring share links, and delete files. SQLite stores account and file metadata, while encrypted file blobs are kept separately.

Problem Statement

Develop a web-based system that protects stored file contents and enables owners to retrieve or temporarily share files without exposing plaintext files directly from storage.

Objectives

• Create a browser-based private file vault.

• Implement registration, login, and logout.

• Encrypt file contents before saving them to disk.

• Restrict private file operations to the owner.

• Support random, time-limited sharing links.

• Document database design and test cases.

Technology Stack

Python, Flask, HTML, CSS, Jinja templates, SQLite, `cryptography` Fernet, Werkzeug password hashing, python-dotenv, Visual Studio Code.

Modules

1. Registration: validates credentials and stores a password hash.

2. Authentication: verifies passwords and uses a signed Flask session.

3. Upload and encryption: encrypts uploaded bytes before writing them to disk.

4. Vault listing: lists only files belonging to the signed-in user.

5. Download: checks ownership, decrypts in memory, and returns an attachment.

6. Sharing: creates random bearer tokens and expiry timestamps; stores only token digests.

7. Shared download: rejects invalid or expired links.

8. Delete: removes the encrypted blob and associated share links.

Architecture

Browser → Flask routes → authentication/ownership checks → SQLite metadata database and encrypted-file storage. The security helpers handle password hashing, encryption, decryption, and token digesting.

Database

• `users(id, username, password_hash, created_at)`

• `files(id, owner_id, original_name, stored_name, size_bytes, created_at)`

• `shares(id, file_id, token_hash, expires_at, created_at)`

A user may own many files. A file may have many share links.

Security Design

Fernet provides symmetric authenticated encryption for stored file bytes. Passwords are hashed using Werkzeug's scrypt-based password hashing. Share tokens use a cryptographically secure random generator; only their SHA-256 digest is stored. Private download, share creation, and delete routes check file ownership.

Requirements

Hardware: dual-core CPU, 4 GB RAM recommended, adequate disk space. Software: Python 3.10+, pip, a modern browser, and Windows/macOS/Linux.

Limitations

This is an educational prototype, not a production-ready storage platform. It lacks CSRF middleware, rate limiting, malware scanning, multi-factor authentication, key rotation, audit logging, and independent security review. The server holds the encryption key and can decrypt files, so this is not end-to-end encryption. Files are buffered in memory and have a configurable size limit.

Future Enhancements

Add CSRF protection, rate limiting, MFA, per-file keys, key rotation, cloud storage, audit logs, recipient verification, share revocation UI, chunked upload, and security testing.

Conclusion

CipherVault demonstrates web development, database design, password hashing, symmetric encryption, authorization checks, and temporary file sharing in a medium-level BCA project.